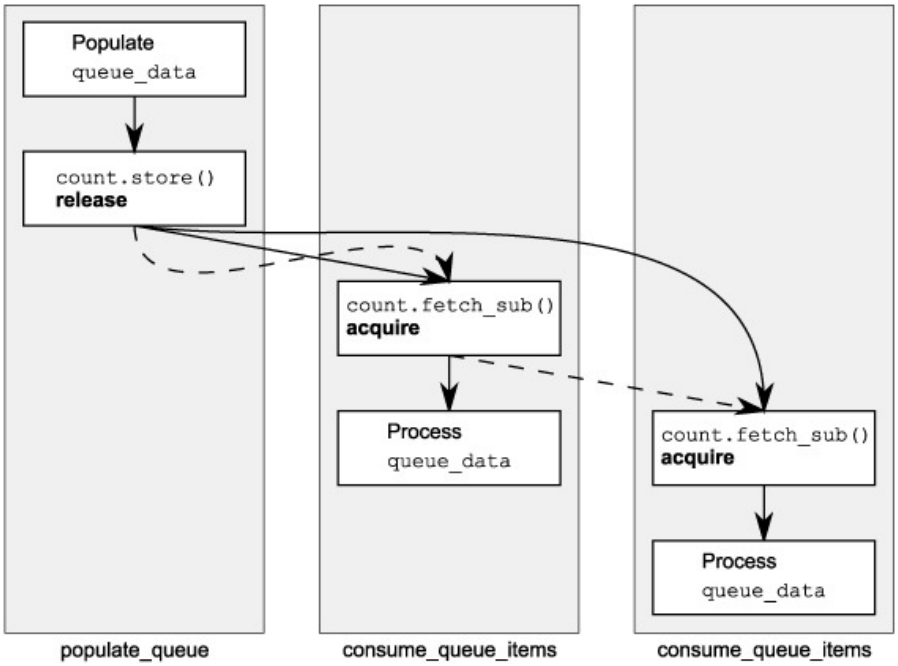
wouldn't be safe to read the shared buffer unless the first `fetch_sub()` also had `memory_order_release` semantics, which would introduce unnecessary synchronization between the two consumer threads. Without the release sequence rule or `memory_order_release` on the `fetch_sub` operations, there would be nothing to require that the stores to the `queue_data` were visible to the second consumer, and you would have a data race. Thankfully, the first `fetch_sub()` *does* participate in the release sequence, and so the `store()` synchronizes-with the second `fetch_sub()`. There's still no synchronizes-with relationship between the two consumer threads. This is shown in figure 5.7. The dotted lines in figure 5.7 show the release sequence, and the solid lines show the happens-before relationships.

Figure 5.7. The release sequence for the queue operations from listing 5.11



There can be any number of links in the chain, but provided they're all read-modify-write operations such as `fetch_sub()`, the `store()` will still synchronize-with each one that's tagged `memory_order_acquire`. In this example, all the links are the same, and all are acquire operations, but they could be a mix of different operations with different memory-ordering semantics.

Although most of the synchronization relationships come from the memory-ordering semantics applied to operations on atomic variables, it's also possible to introduce additional ordering constraints by using *fences*.

5.3.4. Fences

An atomic operations library wouldn't be complete without a set of fences. These are operations that enforce memory-ordering constraints without modifying any data and are typically combined with atomic operations that use the `memory_order_relaxed` ordering constraints.